Math. Log. Quart. 39 (1993), 133 - 142

Mathematical Logic Quarterly

A Formalization of Sambins's Normalization for GL

Edward Hermann Haeusler and Luiz Carlos Pereira

Department of Computer Science and Department of Philosophy,
Pontifícia Universidade Católica do Rio de Janeiro,
Rua Marques de São Vicente, 225, Gávea, Rio de Janeiro, RJ
CEP 22453, Brazil[1)]

Abstract

Sambin [6] proved the normalization theorem (Hauptsatz) for GL, the modal logic of provability, in a sequent calculus version called by him GLS. His proof does not take into account the concept of reduction, commonly used in normalization proofs. Bellini [1], on the other hand, gave a normalization proof for GL using reductions. Indeed, Sambin's proof is a decision procedure which builds cut-free proofs. In this work we formalize this procedure as a recursive function and prove its recursiveness in an arithmetically formalizable way, concluding that the normalization of GL can be formalized in PA.
MSC: 03F05, 03B35, 03B45.
Key words: Sequent calculus, cut-elimination, modal logic of provability, normalization of proofs, mechanization of proof procedures.

1. The sequent calculus GLS for GL

The system GL (Gödel–Löb) is a propositional modal logic, defined as follows:

1. all tautologies are axioms,
2. $\vdash_{GL} \Box(A \to B) \to (\Box A \to \Box B)$,
3. $\vdash_{GL} \Box A \to \Box\Box A$,
4. $\vdash_{GL} \Box(\Box A \to A) \to \Box A$.

Proofs are constructed by using the rules Modus Ponens and Necessitation (rule G). We recall that the rule G says: if $\vdash_{GL} A$ then $\vdash_{GL} \Box A$.

In the arithmetic interpretation, propositional letters are mapped into sentences of PA (Peano Arithmetic) and $\Box$ is taken to be the provability predicate *Bew*. This way the item 4 above is viewed as LÖB's theorem (see [4]).

[1)]The work of the first author was supported by SID Informatica. We thank Prof. ANDRES RAGGIO who gave a short course about GL at PUC/RJ. We also thank MURILO FRANCA who built a prototype of an automatic theorem prover to GL by implementing the function discussed here in LISP.

SAMBIN's version of GL in sequent calculus, called GLS, has as its main rule GLR:

$$\frac{\Delta, \Box\Delta, \Box\gamma \Rightarrow \gamma}{\Box\Delta \Rightarrow \Box\gamma}.$$

However, he also allows the use of a variation of it which includes thinning rules:

$$\frac{\Delta, \Box\Delta, \Box\gamma \Rightarrow \gamma}{\Theta, \Box\Delta \Rightarrow \Box\gamma, \Xi}.$$

This last version is used in his normalization theorem. But we can see that is only a matter of naming since it can be deduced from the former. The GLS system then has the traditional rules of the classical sequent calculus plus GLR. However, it also includes the $\bot$, and the rules which are not double-sound ($\vee$-right and $\wedge$-left) are replaced by

$$\frac{\Delta \Rightarrow \Gamma, \gamma_1, \gamma_2}{\Delta \Rightarrow \Gamma, \gamma_1 \vee \gamma_2}, \quad \frac{\delta_1, \delta_2, \Delta \Rightarrow \Gamma}{\delta_1 \wedge \delta_2, \Delta \Rightarrow \Gamma}.$$

Thus, thinning rules are not needed any more and the rules for negation are instances of the rules for implication, for $\neg A = A \rightarrow \bot$.

The axioms of GLS (initial sequents) have the form $\Delta \Rightarrow \Gamma$, where either $\Delta \cap \Gamma \neq \emptyset$ or $\bot \in \Delta$.

Later we shall introduce the interchange rules in GLS. The resulting system is called GLS'. Note that the cut-rule is not included in GLS.

2. The decision procedure

SAMBIN's procedure falls into two parts: the first builds up a cut-free proof of a given formula if it has some proof, and the other part builds up a countermodel to the sequent if it does not have any proof. The countermodel construction is done from the attempts done by the proof construction part in proving the sequent. Notice that this procedure follows the style of BETH's completeness proof. Our formalization is done by two functions and each of them formalizes one of the parts mentioned above.

The function *dec* is responsible for the cut-free proof construction. Since we need to keep track of the attempts of proof (represented by se- quent trees), the result of *dec* for a given sequent is a sequent tree together with the information about whether the sequent tree is a proof or not. Thus,

$$dec : S_{GL} \longrightarrow D_{GLS'} \times \{0, 1\},$$

where S_{GL} is the set of sequents in the language of GL and $D_{GLS'}$ is the set of trees of sequents in the language of GL. One can notice that the set of deductions is contained in $D_{rmDLS'}$.

The function *Cmodel* is responsible for the countermodel construction. It yields a terminal Kripke frame as countermodel to the sequent given to *dec* from the tree generated by it. Thus,

$$Cmodel : D_{GLS'} \longrightarrow Kmo,$$

where *Kmo* is the set of codes of terminal Kripke frames, which is really the set of codes of finite trees. Reminding that the former can be seen as finite trees.

However, normalization is a procedure that yields a normal proof from a given proof. In our case we do not need to take into account the whole proof,but only its conclusion. Since there is a proof of the conclusion (the given proof) the procedure when applied to it will have as result the desired normal proof. The role played by *Cmodel* is to assure that *dec* will not work out only when the sequent is not valid, thus if it has a proof, a normal proof will be reached.

Thus the function that formalizes the normalization is defined as

$$Normal\left(\begin{array}{c}\Pi\\ \Delta \Rightarrow \Gamma\end{array}\right) = \pi_1(dec(\Delta \Rightarrow \Gamma)).$$

In order to have the normalization formalized in PA, *Normal* must be recursive. But, getting the conclusion of a given proof is a recursive process, as well the first projection π_1. Thus it remains to prove the recursiveness of *dec*. Besides, such proof must be carried out within PA itself. Since *Cmodel* ensures partially the correctness of *dec*, its recursiveness is equally important. We remind that *dec* itself also must assure its correctness, because of this we used the word "partially". It is clear that the proof of the recursiveness of *Cmodel* also must be carried out within PA.

3. The function *dec*

SAMBIN's construction is mainly based on the use of double-sound rules, so ensuring the countermodel construction from an unsuccessfull proof search can be obtained. This is very similar to SMULLYAN's [7] search for an atomically closed tableaux, viewed from a sequent calculus viewpoint. The difference is that in GL we stop the analysis when we reach a modal formula in the antecedent of a sequent. That is equivalent to give the status of atomic formulas to modal formulas signed with T. FITTING [3] gave a complete definition of a tableaux system for GL just in this way. We would like to comment that the use of double-sound rules is in a certain way equivalent to the use of Hintikka sets in completeness proofs.

We can see from the above comments that any sequent that contains in its antecedent only modal and atomic formulas and in its consequent only atomic formulas cannot be analysed any more. Thus, in this case we already have conditions to decide whether the sequent has a proof or not. The decision is as follows:

- If the sequent has common elements between the antecedent and the succedent, then it has a proof (itself), and hence it is valid.
- If it has not common elements between the antecedent and the succedent, then a terminal Kripke frame that falsifies it can be constructed. Indeed, such frame needs only one world.

The function *dec* is defined by recursion. Thus, we can consider the above decision as the basic step of it. It is clear that in the first case the value resulting is 1 (there is a cut–free proof) and in the second the value is 0 (there is not a cut–free proof). Besides, in any case the sequent itself, as representing the tree of sequents, is also the result of applying *dec* to it. Well, the other cases are done by analysis of the formulas belonging to the sequent. In order to obtain the uniqueness of a function,

dec operates the analysis over the extreme formulas in each part of the sequent, i.e. it analyses the leftmost formula of the antecedent and the rightmost formula of the succedent. Besides, any analysis on the succedent has priority over that on the antecedent.

Because of the explained above we have to introduce the exchange rule in GLS', for we need to consider another formula in the consequent (or antecedent) when we cannot analyse the current one any more.

For the sake of brevity (and clarity) we show only the items of *dec* that we consider most important, in explaining how *dec* works out.

1. $dec(\Delta \Rightarrow \Gamma) = (\Delta \Rightarrow \Gamma, 1)$ iff $\Delta \cap \Gamma \neq \emptyset$ or $\bot \in \Delta$.

2. $dec(\Delta \Rightarrow \Gamma) = (\Delta \Rightarrow \Gamma, 0)$ iff $\Delta \cap \Gamma = \emptyset$, $\bot \notin \Delta$, and besides all members of Γ are sentential letters and all members of Δ are either sentential letters or modalized formulas.

3. If $\Delta = \Delta_1, \Box\Delta_2$ and $\Gamma = \Gamma_1, \Box\Gamma_2$, where $\Delta_1 \cap \Gamma_2$ contains only sentential letters, then

$$dec(\Delta \Rightarrow \Gamma) = \left(\dfrac{\dfrac{\Pi}{\Delta_2, \Box\Delta_2, \Box\gamma_i \Rightarrow \gamma_i}}{\dfrac{\Box\Delta_2 \Rightarrow \Box\gamma_i}{\Delta_1, \Box\Delta_2 \Rightarrow \Box\Gamma_1, \Gamma_2}} \ , \ 1 \right)$$

iff $\Gamma_1 = \{\gamma_1, \ldots, \gamma_k\}$ and i is the least $i \in \{1, \ldots, k\}$ such that

$$dec(\Delta_2, \Box\Delta_2, \Box\gamma_i \Rightarrow \gamma_i) = \left(\dfrac{\Pi}{\Delta_2, \Box\Delta_2, \Box\gamma_i \Rightarrow \gamma_i} \ , \ 1 \right).$$

Otherwise

$$dec(\Delta \Rightarrow \Gamma) = \left(\dfrac{\dfrac{\Pi_1}{\Delta_2, \Box\Delta_2, \Box\gamma_1 \Rightarrow \gamma_1} \quad \ldots \quad \dfrac{\Pi_k}{\Delta_2, \Box\Delta_2, \Box\gamma_k \Rightarrow \gamma_k}}{\dfrac{\Box\Delta_2 \Rightarrow \Box\gamma_1}{\Delta_1, \Box\Delta_2 \Rightarrow \Box\Gamma_1, \Gamma_2}} \ , \ 0 \right),$$

where

$$dec(\Delta_2, \Box\Delta_2, \Box\gamma_i \Rightarrow \gamma_i) = \left(\dfrac{\Pi_i}{\Delta_2, \Box\Delta_2, \Box\gamma_i \Rightarrow \gamma_i} \ , \ 0 \right),$$

for $i = 1, \ldots, k$.

4. If $\Delta_1 = \delta_1, \delta_2, \Delta_1, \Box\Delta_2$ and δ_1 is a sentential letter or a modal formula, and there is at least one non-modal formula between any two modal formulas in Δ, then

$$dec(\Delta \Rightarrow \Gamma) = \left(\dfrac{\dfrac{\Pi}{\delta_2, \Delta_1, \delta_1, \Box\Delta_2 \Rightarrow \Gamma}}{\delta_1, \delta_2, \Delta_1, \Box\Delta_2 \Rightarrow \Gamma} \ , \ r \right),$$

where

$$dec(\delta_2, \Delta_1, \delta_1, \Box\Delta_2 \Rightarrow \Gamma) = \left(\dfrac{\Pi}{\delta_2, \Delta_1, \delta_1, \Box\Delta_2 \Rightarrow \Gamma} \ , \ r \right).$$

5.

$$dec(\delta_1 \to \delta_2, \Delta_1 \Rightarrow \Gamma) = \left(\frac{\begin{array}{cc} \Pi_1 & \Pi_2 \\ \Delta_1 \Rightarrow \delta_1, \Gamma & \delta_2, \Delta_1 \Rightarrow \Gamma \end{array}}{\delta_1 \to \delta_2, \Delta_1 \Rightarrow \Gamma} \ , \ r \right),$$

where

$$dec(\Delta_1 \Rightarrow \delta_1, \Gamma) = \left(\begin{array}{c} \Pi_1 \\ \Delta_1 \Rightarrow \delta_1, \Gamma \end{array} \ , \ r_1 \right), \ dec(\delta_2, \Delta_1 \Rightarrow \Gamma) = \left(\begin{array}{c} \Pi_2 \\ \delta_2, \Delta_1 \Rightarrow \Gamma \end{array} \ , \ r_2 \right),$$

and if $r_1 = r_2 = 1$, then $r = 1$, else $r = 0$.

6.

$$dec(\delta_1 \vee \delta_2, \Delta_1 \Rightarrow \Gamma) = \left(\frac{\begin{array}{cc} \Pi_1 & \Pi_2 \\ \delta_1, \Delta_1 \Rightarrow \Gamma & \delta_2, \Delta_1 \Rightarrow \Gamma \end{array}}{\delta_1 \vee \delta_2, \Delta_1 \Rightarrow \Gamma} \ , \ r \right),$$

where

$$dec(\delta_1, \Delta_1 \Rightarrow \Gamma) = \left(\begin{array}{c} \Pi_1 \\ \delta_1, \Delta_1 \Rightarrow \Gamma \end{array} \ , \ r_1 \right), \ dec(\delta_2, \Delta_1 \Rightarrow \Gamma) = \left(\begin{array}{c} \Pi_2 \\ \delta_2, \Delta_1 \Rightarrow \Gamma \end{array} \ , \ r_2 \right),$$

and if $r_1 = r_2 = 1$, then $r = 1$, else $r = 0$.

7.

$$dec(\Delta \Rightarrow \Gamma_1, \gamma_1 \vee \gamma_2) = \left(\frac{\begin{array}{c} \Pi \\ \Delta \Rightarrow \Gamma_1, \gamma_1, \gamma_2 \end{array}}{\Delta \Rightarrow \Gamma_1, \gamma_1 \vee \gamma_2} \ , \ r \right),$$

where

$$dec(\Delta \Rightarrow \Gamma_1, \gamma_1, \gamma_2) = \left(\begin{array}{c} \Pi \\ \Delta \Rightarrow \Gamma_1, \gamma_1, \gamma_2 \end{array} \ , \ r \right).$$

8.

$$dec(\Delta \Rightarrow \Gamma_1, \gamma_1 \to \gamma_2) = \left(\frac{\begin{array}{c} \Pi \\ \gamma_1, \Delta \Rightarrow \Gamma_1, \gamma_2 \end{array}}{\Delta \Rightarrow \Gamma_1, \gamma_1 \to \gamma_2} \ , \ r \right),$$

where

$$dec(\gamma_1, \Delta \Rightarrow \Gamma_1, \gamma_2) = \left(\begin{array}{c} \Pi \\ \gamma_1, \Delta \Rightarrow \Gamma_1, \gamma_2 \end{array} \ , \ r \right).$$

We remind the reader that the items of definition of *dec* operating on the succedent have priority over the ones that operate on the antecedent. Thus, for example, *dec* first applies the interchange rule to the succedent (an item not written down here) when it can chose between the two cases. However, this can only happen when no other rule, including the GLR, can be applied. Thus, observing the definition of *dec*, we notice that all items have empty domain intersection each other, looking to them as functions. Besides, taking into account what we have explained above about how to solve indeterminism, we have then the following proposition:

Proposition 3.1. *If dec is defined for a sequent* $\Delta \Rightarrow \Gamma$, *then* $dec(\Delta \Rightarrow \Gamma)$ *is unique.*

It is clear that *dec* is a partial recursive function, since all procedures employed in its definition are computable (the reader is invited to check this). However, the question remains as to whether *dec* is a total function. In the next section we prove that *dec* is really a recursive (total) function.

4. The recursiveness of *dec*

We want prove the following

Theorem 4.1. *For any sequent* $\Delta \Rightarrow \Gamma$,

$$\pi_2(dec(\Delta \Rightarrow \Gamma)) = 1 \text{ or } \pi_2(dec(\Delta \Rightarrow \Gamma)) = 0.$$

We prove the theorem by induction. The induction value must mirror the recursive feature of *dec*. We notice that we have three kinds of rules: the interchange rules, the propositional rules, and the GLR. By looking at the recursive structure of *dec* we see that here is a hierarchy among these inference rules, that is:

- The interchange rules prepare the sequent to the application of the propositional and GLR rules. Since the propositional rules are applied only on the extremes of the sequent, and the GLR rule needs it free of propositional formulas.
- The propositional rules prepare the sequent to the GLR application.

Thus, GLR applications have the highest level. Therefore, according to the above, we can think of our induction value as being a lexicographic value (i,ii,iii), where i estimates the number of GLR applications for a given sequent in the longest possible sequence of evaluations of *dec*, ii estimates the number of propositional rules applications between each GLR application, and iii estimates the number of interchange applications between each group of propositional rules applications. We can see that the sum of the degrees of the non-modal formulas in a sequent can be the value ii. For iii we can consider a value that measures the distance between the extreme of the antecedent and the nearest modal formula in it (with respect to this extreme) plus the same done with respect to the succedent. This value takes into account the number of modal formulas already put together in a sublist contained in the part of the sequent (antecedent or succedent). Thus, take into account only these two values and do not consider the GLR applications, i.e. each evaluation of an GLR application, where necessary, is done by using intems 1 or 2 of the definition of *dec*. Call this new function dec^*. Then we have the following

Fact 1. *Let* dec^* *be the function dec without taking into account the item* 3 *of the definition of dec, then* dec^* *is defined to any* $\Delta \Rightarrow \Gamma$.

Proof. By induction on the value (ii,iii), where ii is the sum of $g(F)$ (degree of F) for each $F \in \Delta \cup \Gamma$ which is not a modal formula (m-formula), and iii is $n \cdot \text{iii}_2 + \text{iii}_1$, where n is the number of formulas in $\Delta \cup \Gamma$, iii_1 is the number of formulas between the beginning of Δ and the first formula in Δ which is neither an m-formula nor a sentential letter plus the number of formulas between the end of Γ and the last formula in Γ which is neither an m-formula nor a sentential letter, and iii_2 is the number of m-formulas in $\Delta \cup \Gamma$ minus the length of the sublist of m-formulas at the end of Δ and the length of the sublist of m-formulas at the beginning of Γ.

In order to estimate the number of GLR applications of the longest sequence of evaluation of *dec* we define the function *md* that mirrors *dec* w.r.t. the item 3 of its definition. We notice that this item is only defined for a special kind of sequent, obtained by the function *dec* itself by means of its propositional part, i.e. dec^*. It is clear that if for a given sequent the second element of the dec^*-result is 1, then it is valid. Hence the number of GLR applications equals to 0. Otherwise, there must be non-axioms as top-sequents in the deduction (tree) constructed by dec^*. Then *dec* would apply the GLR rule to these sequents in order to continue the evaluation. Thus, in order to obtain the number of GLR applications we must see what happens in each alternative of evaluation and naturally what is the possible maximal number of GLR applications in the longest of these alternatives.

We use the function *Hyp* in order to get the set of top-sequents that are not axioms. Clearly *Hyp* has a deduction as argument. In order to improve the readability of the definition of *md* we define an function md^* which, for a given sequent, computes (estimates) the number of GLR applications for one of these sequents that would continue the *dec*-evaluation. In order to avoid any confusion about the apparent mutual recursion present in the definition of md^*, one can thought of md^* as being replaced by its definition in the definition of *md*, i.e., it can be thought as a macro. So

$$md^*(\Theta, \Box\Delta \Rightarrow \Box\Gamma, \Lambda) = \begin{cases} 0 & \text{if} \quad \Gamma = \emptyset, \\ \max_{\gamma\in\Gamma}(md(\Delta, \Box\Delta, \Box\gamma \Rightarrow \gamma)) & \text{if} \quad \Gamma \neq \emptyset. \end{cases}$$

Therefore, the number of applications of GLR in the longest evaluation sequence (with respect to GLR) is given by the greatest possible number of GLR applications in each alternative path. Thus,

$$md(\Xi_1, \Box\Xi_2, \Rightarrow \Box\Upsilon_2, \Upsilon_1) = 0 \text{ iff } \pi_2(dec^*(\Xi_1, \Box\Xi_2 \Rightarrow \Box\Upsilon_2, \Upsilon_1)) = 1,$$

and otherwise

$$md(\Xi_1, \Box\Xi_2 \Rightarrow \Box\Upsilon_2, \Upsilon_1) = \max\{md^*(\Theta, \Box\Delta \Rightarrow \Gamma, \Lambda) : \Theta, \Box\Delta \Rightarrow \Gamma, \Lambda \in Hyp(\pi_1(dec^*(\Xi_1, \Box\Xi_2 \Rightarrow \Box\Upsilon_2, \Upsilon_1)))\} + 1.$$

In order to analyse how *dec* works we define the concept of an *ant*-$\Box$ subformula of a formula F as being any subformula F' of F such that F' is in F not under the scope of a $\Box$, and F' is a subformula of an implication and this implication is not a subformula of an antecedent of an implication. The goal of this concept intends to attain the modal subformulas of a formula belonging to the antecedent of a sequent, that will belong to the succedent of such sequent when item 5 of the definition of *dec* is applied by dec^* to it. There is a similar concept related to the negation, reminding that $\neg A = A \rightarrow \bot$. For the sake of attaining the modal subformulas of a formula belonging to the succedent of a sequent that will still belong to it after dec^* application (look to item 8 of the definition of *dec*), we define the dual concept of *ant*-$\Box^*$ subformula. We also notice that an *ant*-$\Box$ subformula belonging to the succedent of a sequent will belong to the antecedent of it after dec^* application.

Looking to the definition of *md* we notice that any formula belonging to $\Box\Xi_2$ is either an *ant*-$\Box$ subformula of some formula of Ξ_1 or an *ant*-$\Box^*$subformula of some formula of Υ_1 or even belongs to $\Box\Upsilon_2$. Thus, a modal subformula that had already

been in the succedent and is an *ant*-$\Box$ subformula of some formula in the antecedent is during the *md*-evaluation in a branch of the *md*-evaluation that will result in value 0, for it will be at the same time in the succedent (because of *dec*-effect) and in the antecedent (because of md^*). Thus, it cannot be the case of repeating the choice of modal formulas (more than twice) to GLR application in a branch of *md*-evaluation. This means that the alternatives of choice of modal formulas in order to continue *md*-evaluation are always decreasing from one step to other. An important consequence is that *md* continues by choosing modal formulas that are subformulas of an already choosen. Thus the degree of the modal formulas choosen to GLR application is less than the one choosen previously (w.r.t. *md*-evaluation). So there is a step during the *md*-evaluation when there is no more modal formulas and hence the evaluation finishes. Therefore, we have the following result:

Lemma 4.1. *For every sequent* $\Xi_1, \Box\Xi_2 \Rightarrow \Box\Upsilon_2, \Upsilon_1$, $md(\Xi_1, \Box\Xi_2 \Rightarrow \Box\Upsilon_2, \Upsilon_1)$ *is a natural number.*

Therefore, *md* is a recursive function and hence can be formalized in PA, and the proof of recursiveness, mainly that *md* is total, can be carried out within PA, since it is done by induction and using an auxiliary lemma about the impossibility of repeating modal formulas more than twice for GLR applications.

Now, we can prove the desired result (Theorem 4.1) by induction on the value (i,ii,iii), where i=$md(\Delta \Rightarrow \Gamma)$ and ii and iii are as explained in Fact 1. $\Box$

We note that even if the proof of the recursiveness of *md* were not arithmetically formalizable the above result about the formalization in PA of the proof of the recursiveness of *dec*, i.e., that *dec* is a total function, would still hold. For, the proof that it is a total function is done in the metalanguage level. We would like to emphasize that our proof of the recursiveness was done in two parts, the construction of the partial-recursive function, where the proof of its partial-recursiveness is assured by the construction itself, and the proof that it is a total function. Even this last part needs a proof of its formalization in PA.

In the next section we prove the correctness of *dec*, which is the possibility of the countermodel construction when *dec* does not have a cut-free proof as result.

5. The function *Cmodel*

We begin this section by completing the definition of *Cmodel* as stated in the beginning of Section 2. Firstly, we define the set of Kripke terminal frames always thinking about its codification in number theory (PA). Thus, *Kmo* is the set of Kripke terminal frames *mo* such that $U = \bigcup_{i=0}^{\infty} U_i$, where $U_0 = \{root\}$, $U_{i+1} = \{x \cdot j : x \in U_i,\ j \in \mathsf{N}\}$, where "$\cdot$" denotes the concatenation of symbols and N is the set of natural numbers interpreted as a denumerable alphabet. Then $mo = \langle U', R, \alpha\rangle$, where $U' \subseteq U$, $R \subseteq U' \times U'$, $\alpha : V \longrightarrow 2^{U'}$, where V is the set of sentential letters of the language of GLS'. One could point out that the function α cannot be coded into the natural numbers, but as we will see later it is enough its definition to a finite subset of V. Thus, we can code α. For the sake of a better formalization think about it as a partial function.

We recall that a double-sound rule is such that the conclusion is false if and only if at least one of its premises is false, too. The result of *Cmodel* is a countermodel

to the conclusion of its argument. Recall that its argument is a tree of sequents and the root can be viewed as the conclusion of this tree in the context of deductions. As we have already said *Cmodel* works by building of countermodels from countermodels to the top-sequents which are not axioms. As all propositional rules of GLS' are double-sound we assure the unsatisfiability of the conclusion. We would like point out that the construction in item 3 of the definition of *dec* is not really an GLR application. It is rather a way of putting together all the unsuccessfull attempts to prove a premise of a true GLR rule. However, we call it GLR application for want of a better name. In the case of such GLR application we construct a countermodel based on the countermodels constructed to each one of its premises. However, in the propositional cases some premises may not be unsatisfiable. So, in order to sort out this problem we define *Cmodel* depending on the function C which has a pair (K, t) as value, where K is a Kripke frame and t tell us whether K is a countermodel to the argument of C or not. Thus,

$$Cmodel\left(\begin{array}{c}\Pi\\ \Delta \Rightarrow \Gamma\end{array}\right) = \pi_1\left(C\left(\begin{array}{c}\Pi\\ \Delta \Rightarrow \Gamma\end{array}, root\right)\right).$$

Since the construction is recursive and we must guarantee that the worlds constructed previously are distinct we give to C the name of the world (in the language U) that must be the root of the terminal Kripke frame which can be viewed as a finite tree. Thus, it can be understood that U is the name of the root of any frame, U_1 is the set of the names of its immediate successors, and in general U_{i+1} is the set of names of the immediate successors of the elements labeled by U_i. There is no order among the members of U_i, the natural numbers are used only to distinguish them. Thus, $C : D_{\mathrm{GLS}'} \times U \longrightarrow Kmo \times \{0, 1\}$ is defined by the following conditions:

If $\Delta_1 \cup \Gamma$ contains only sentential letters and $\Delta_1 \cap \Gamma = \emptyset$, then

$$C(\Delta_1, \Box\Delta_2 \Rightarrow \Gamma, r) = \{(\{r\}, \emptyset, \alpha\rangle, 1),$$

i.e., $\alpha(A) = \{r\}$ for all $A \in \Delta_1$, and $\alpha(A) = \emptyset$ for all A not belonging to Δ_1.

The propositional rule cases are treated by getting the countermodel to the left premise unless it is not a countermodel. Recall this is given by the second element of the result of C. However, if both premises are valid, then C returns the pair $(\langle\emptyset, \emptyset, \alpha \longrightarrow \{\emptyset\}\rangle, 0)$ as result.

The GLR case is treated as follows:

$$C\left(\begin{array}{c}\begin{array}{ccc}\Pi_1 & & \Pi_k\\ \dfrac{\Delta_2, \Box\Delta_2, \Box\gamma_1 \Rightarrow \gamma_1}{\Box\Delta_2 \Rightarrow \Box\gamma_1} & \cdots & \dfrac{\Delta_2, \Box\Delta_2, \Box\gamma_k \Rightarrow \gamma_k}{\Box\Delta_2 \Rightarrow \Box\gamma_k}\end{array}\\ \hline \Delta_1, \Box\Delta_2 \Rightarrow \Box\Gamma_1, \Gamma_2\end{array}, r\right) = (\langle U, R, \alpha\rangle, 1),$$

where

$$C(\Delta_2, \Box\Delta_2, \gamma_i \Rightarrow \gamma_i, r_i) = (\langle U_i, R_i, \alpha_i\rangle, t_i)$$

and

$$U = \bigcup_{i=1}^{k} U_i \cup \{r\}, \quad R = \bigcup_{i=1}^{k} R_i \cup R^*,$$

where $R^* = \{(r, r_i) : i = 1, \ldots, k\}$ and

$$\alpha(A) = \begin{cases} \bigcup_{i=1}^{k} \alpha_i(A) \cup \{r\} & \text{if } A \in \Delta_1, \\ \bigcup_{i=1}^{k} \alpha_i(A) & \text{otherwise.} \end{cases}$$

Notice that this item constructs a new tree that has r as root and all the trees that falsify the respective premise are now subtrees of the new one. So, the resulting tree continues to falsify the premises and besides, it falsifies the conclusion, since its antecedent is true in r (look to the definition of α) and the succedent is false.

Thus, by induction on the number of inference rules in $\pi_1(dec(\Delta \Rightarrow \Gamma))$ we obtain the following result:

Theorem 5.1. *If* $\pi_2(dec(\Delta \Rightarrow \Gamma)) = 0$, *then* $C(\pi_1(dec(\Delta \Rightarrow \Gamma)))$ *is a counter-model to* $\Delta \Rightarrow \Gamma$.

We notice that C is recursive, since all parts of the definition of C are computable. Besides, the above result can be carried out in PA, since it uses induction.

6. Conclusion

The main conclusion of this paper is that we have a finitary proof of the normalization theorem. Thus, we can formalize the normalization theorem for GL in PA. Besides, as consistency is a corollary of the normalization theorem, then GL's consistency theorem can be formalized in PA, too. Another conclusion is that the here formalized decidability process can be viewed as an automatic theorem prover for GL in sequent calculus.

References

[1] Bellini, G., A system of natural deduction for GL. Theoria 7 (1985), 89–114.

[2] Boolos, G., The Unprovability of Consistency: An Essay in Modal Logic. Cambridge University Press, Cambridge 1979.

[3] Fitting, M., Proof Methods for Modal and Intuitionistic Logics. D. Reidel Publ. Comp., Dordrecht 1983.

[4] Leivant, D., On the proof theory of the modal logic for arithmetic provability. J. Symbolic Logic 46 (1981), 531–538.

[5] Prawitz, D., Ideas and results in proof theory. In: Proceedings Second Scandinavian Logic Symposium (J. E. Fenstad, ed.), North-Holland Publ. Comp., Amsterdam 1971.

[6] Sambin, G., and S. Valentini, The modal logic of provability: The sequential approach. Repporto Matematico n. 57 (1982), Università degli Studi di Siena.

[7] Smullyan, R., First Order Logic. Springer-Verlag, Berlin-Heidelberg-New York 1968.

(Received: May 3, 1991)